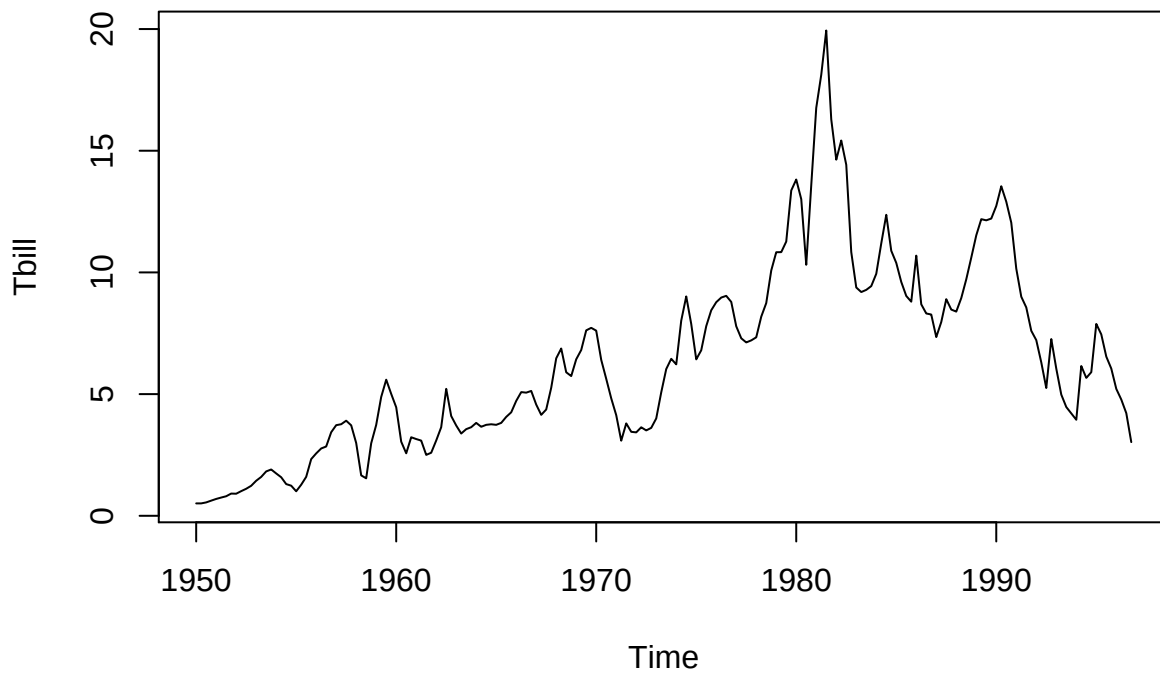


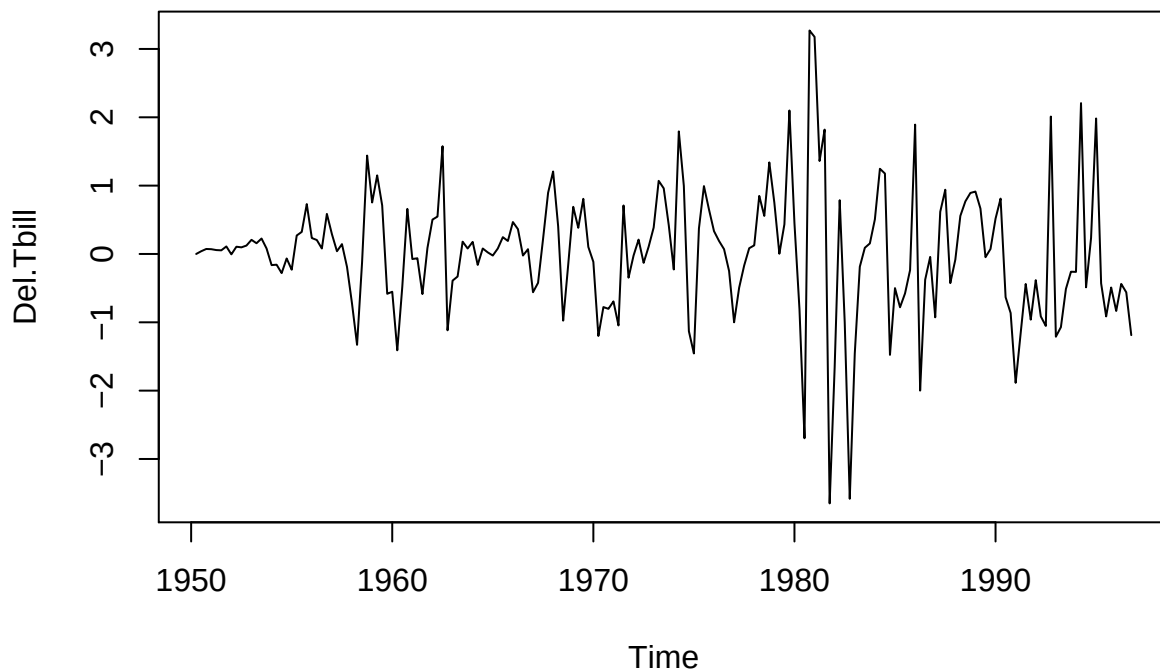
HW4 – Ying Zheng

Part I Problem 1

```
Tbill = Tbrate[,1]
Del.Tbill = diff(Tbill)
#Use both time series and ACF plots
plot(Tbill)
```

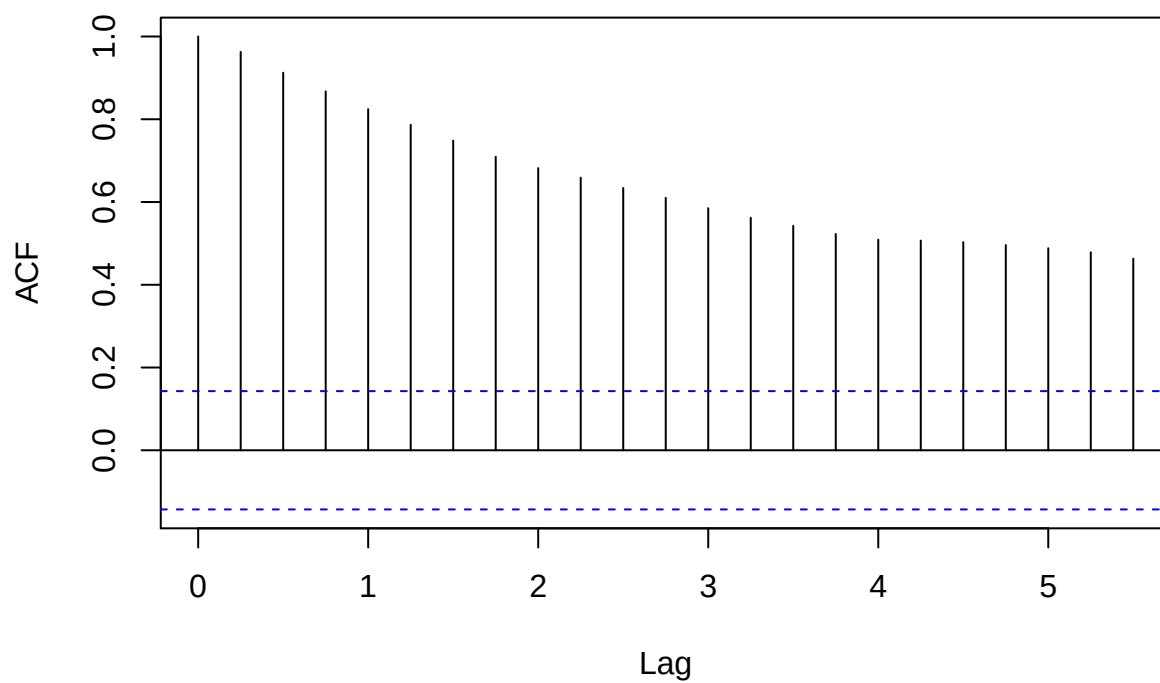


```
plot(Del.Tbill)
```



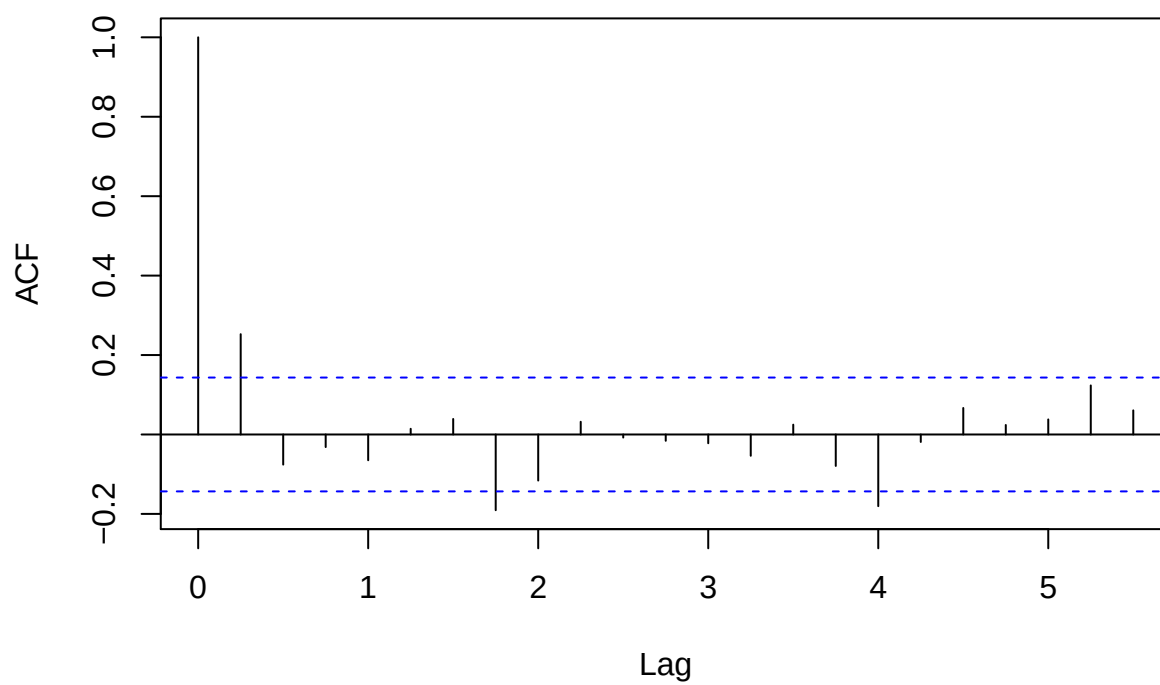
```
acf(Tbill)
```

Series Tbill



```
acf(Del.Tbill)
```

Series Del.Tbill



```

#perform ADF tests on both series
adf.test(as.numeric(Tbill), alternative = "s")

##
## Augmented Dickey-Fuller Test
##
## data: as.numeric(Tbill)
## Dickey-Fuller = -1.925, Lag order = 5, p-value = 0.6075
## alternative hypothesis: stationary

adf.test(as.numeric(Del.Tbill), alternative = "s")

## Warning in adf.test(as.numeric(Del.Tbill), alternative = "s"): p-value
## smaller than printed p-value

##
## Augmented Dickey-Fuller Test
##
## data: as.numeric(Del.Tbill)
## Dickey-Fuller = -5.2979, Lag order = 5, p-value = 0.01
## alternative hypothesis: stationary

```

By looking at time-series plots, Del.Tbill is more stationary. ACF plot suggests no correlation after lag one for Del.Tbill while suggests strong correlation in all lags for Tbill. P-value of ADF test is 0.01 (<0.05) for Del.Tbill, which indicates zero correlation, while p-value of ADF test is 0.6075 for Tbill. Del.Tbill shows conditional hetskedasticity, since variance seems to grow larger and larger over time.

Problem 2 (a)

```

garch.model.Tbill = garchFit(formula = ~arma(1, 0) + garch(1, 0), Del.Tbill)

##
## Series Initialization:
## ARMA Model: arma
## Formula Mean: ~ arma(1, 0)
## GARCH Model: garch
## Formula Variance: ~ garch(1, 0)
## ARMA Order: 1 0
## Max ARMA Order: 1
## GARCH Order: 1 0
## Max GARCH Order: 1
## Maximum Order: 1
## Conditional Dist: norm
## h.start: 2
## llh.start: 1
## Length of Series: 187
## Recursion Init: mci
## Series Scale: 0.9422364
##
## Parameter Initialization:
## Initial Parameters: $params
## Limits of Transformations: $U, $V
## Which Parameters are Fixed? $includes
## Parameter Matrix:
##
##          U          V      params includes
## mu    -0.14290725  0.1429072 0.01197006      TRUE
## ar1    -0.99999999  1.0000000 0.25347489      TRUE

```

```

##      omega  0.00000100 100.0000000 0.10000000    TRUE
##      alpha1 0.00000001  1.0000000 0.10000000    TRUE
##      gamma1 -0.99999999  1.0000000 0.10000000    FALSE
##      delta  0.00000000  2.0000000 2.00000000    FALSE
##      skew   0.10000000 10.0000000 1.00000000    FALSE
##      shape  1.00000000 10.0000000 4.00000000    FALSE
## Index List of Parameters to be Optimized:
##      mu      ar1  omega alpha1
##      1       2      3      4
## Persistence:                0.1
##
##
## --- START OF TRACE ---
## Selected Algorithm: nlminb
##
## R coded nlminb Solver:
##
## 0:      456.21058: 0.0119701 0.253475 0.100000 0.100000
## 1:      251.86762: 0.0119754 0.236936 1.02079 0.489709
## 2:      250.90930: 0.0120591 0.473682 0.845081 0.893495
## 3:      246.37888: 0.0132650 0.303699 0.757428 1.000000
## 4:      244.30653: 0.0172433 0.228377 0.691915 1.000000
## 5:      240.23776: 0.0270411 0.161803 0.543304 1.000000
## 6:      236.37169: 0.0367592 0.175592 0.370918 1.000000
## 7:      236.31833: 0.0446835 0.210080 0.425054 0.537516
## 8:      235.93894: 0.0447059 0.288586 0.470834 0.622965
## 9:      235.66475: 0.0444909 0.288456 0.377870 0.704783
## 10:     235.36527: 0.0456870 0.217083 0.405800 0.756581
## 11:     235.19182: 0.0473954 0.248964 0.405265 0.772573
## 12:     235.13214: 0.0513735 0.255349 0.382404 0.810199
## 13:     235.08391: 0.0553971 0.257352 0.383930 0.823996
## 14:     234.96674: 0.0712269 0.253195 0.382822 0.857205
## 15:     234.92037: 0.0846539 0.246053 0.380919 0.862586
## 16:     234.91025: 0.0885175 0.242328 0.380676 0.848471
## 17:     234.90819: 0.0890151 0.241257 0.380809 0.836077
## 18:     234.90814: 0.0886908 0.241545 0.380857 0.834926
## 19:     234.90814: 0.0886149 0.241631 0.380886 0.834829
## 20:     234.90814: 0.0886153 0.241635 0.380889 0.834828
##
## Final Estimate of the Negative LLH:
## LLH: 223.7818      norm LLH: 1.196694
##      mu      ar1      omega      alpha1
## 0.08349652 0.24163450 0.33815662 0.83482811
##
## R-optimhess Difference Approximated Hessian Matrix:
##      mu      ar1      omega      alpha1
## mu      -379.01802 -68.15986 -37.51437  4.76325
## ar1      -68.15986 -209.40200  51.00153  8.23281
## omega    -37.51437  51.00153 -399.68185 -54.31253
## alpha1    4.76325  8.23281 -54.31253 -24.58337
## attr("time")
## Time difference of 0.05174804 secs
##
## --- END OF TRACE ---

```

```
##
##
## Time to Estimate Parameters:
## Time difference of 0.0914731 secs

summary(garch.model.Tbill)

##
## Title:
## GARCH Modelling
##
## Call:
## garchFit(formula = ~arma(1, 0) + garch(1, 0), data = Del.Tbill)
##
## Mean and Variance Equation:
## data ~ arma(1, 0) + garch(1, 0)
## <environment: 0x7fe18ec34168>
## [data = Del.Tbill]
##
## Conditional Distribution:
## norm
##
## Coefficient(s):
##      mu      ar1      omega    alpha1
## 0.083497 0.241635 0.338157 0.834828
##
## Std. Errors:
## based on Hessian
##
## Error Analysis:
##      Estimate Std. Error t value Pr(>|t|)
## mu      0.08350    0.05391   1.549 0.121395
## ar1      0.24163    0.07280   3.319 0.000902 ***
## omega    0.33816    0.06145   5.503 3.73e-08 ***
## alpha1   0.83483    0.24295   3.436 0.000590 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Log Likelihood:
## -223.7818    normalized: -1.196694
##
## Description:
## Wed Nov 20 16:56:35 2019 by user:
##
##
## Standardised Residuals Tests:
##
##      Jarque-Bera Test  R      Chi^2 26.96616 1.394352e-06
##      Shapiro-Wilk Test R      W      0.957289 1.957134e-05
##      Ljung-Box Test   R      Q(10) 10.8275 0.3711143
##      Ljung-Box Test   R      Q(15) 13.10815 0.5939444
##      Ljung-Box Test   R      Q(20) 16.11144 0.7096868
##      Ljung-Box Test   R^2 Q(10) 12.18313 0.2729873
##      Ljung-Box Test   R^2 Q(15) 14.31743 0.5016035
##      Ljung-Box Test   R^2 Q(20) 17.28754 0.634231
```

```
## LM Arch Test      R      TR^2    9.685432  0.6435358
##
## Information Criterion Statistics:
##      AIC      BIC      SIC      HQIC
## 2.436169 2.505284 2.435279 2.464174
```

```
garch.model.Tbill@fit$matcoef
```

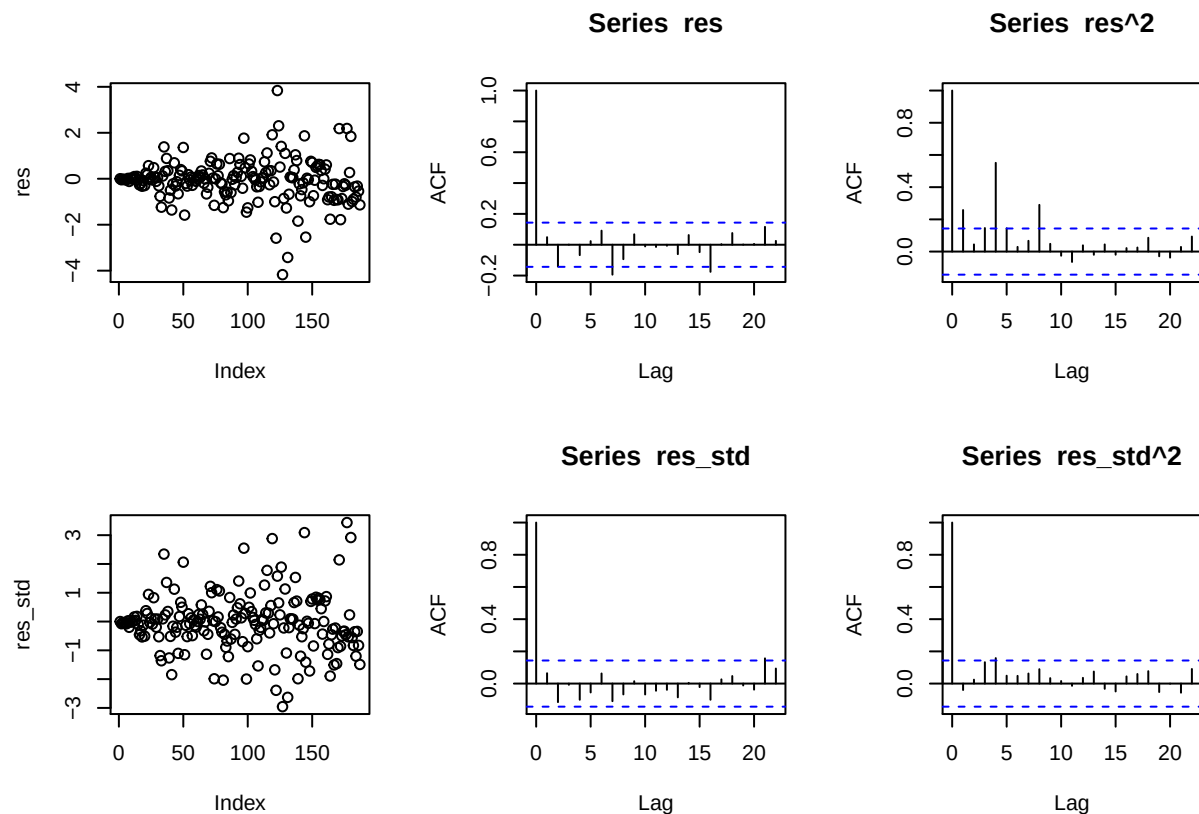
```
##      Estimate Std. Error t value    Pr(>|t|)
## mu      0.08349652 0.05390549 1.548943 1.213955e-01
## ar1      0.24163450 0.07279700 3.319292 9.024598e-04
## omega    0.33815662 0.06144724 5.503203 3.729535e-08
## alpha1   0.83482811 0.24295106 3.436199 5.899382e-04
```

ARMA(1,0) and Garch(1,0) have been fitted.

Problem 2 (b): Estimated $\mu = 0.08349652$, $\text{ar1} = 0.24163450$, $\omega = 0.33815662$, and $\alpha_1 = 0.83482811$.

Problem 3

```
res = residuals(garch.model.Tbill)
res_std = res / garch.model.Tbill@sigma.t
par(mfrow=c(2,3))
plot(res)
acf(res)
acf(res^2)
plot(res_std)
acf(res_std)
acf(res_std^2)
```



(a) $\text{ACF}(\text{res})$ plots the autocorrelation for residuals and meanwhile gives a confidence interval to reject

the null. The above acf plot indicates correlation at lag seven and lag 16, which suggests the model parameters fit quite well.

- (b) $ACF(res^2)$, like $ACF(res)$, instead plots the autocorrelation for residuals squares. The plot implies a volatility clustering, to be sure we have to take further look at residual variance.
- (c) $ACF(res_std^2)$ plots the autocorrelation for residual variance. This plot suggest no correlation in further lags, thus there is no strong evidence of volatility clustering.
- (d) This is an estimate of σ_t .
- (e) It shows lareger variance in largers index.

Problem 4

```
del.log.tbill = diff(log(Tbill))
model = garchFit(formula = ~arma(1, 0) + garch(1, 0), del.log.tbill)
```

```
##
## Series Initialization:
## ARMA Model:          arma
## Formula Mean:        ~ arma(1, 0)
## GARCH Model:         garch
## Formula Variance:    ~ garch(1, 0)
## ARMA Order:          1 0
## Max ARMA Order:      1
## GARCH Order:         1 0
## Max GARCH Order:     1
## Maximum Order:       1
## Conditional Dist:    norm
## h.start:             2
## llh.start:           1
## Length of Series:    187
## Recursion Init:      mci
## Series Scale:        0.1506804
##
## Parameter Initialization:
## Initial Parameters:   $params
## Limits of Transformations: $U, $V
## Which Parameters are Fixed? $includes
## Parameter Matrix:
##           U          V      params includes
## mu      -0.63215833  0.6321583 0.05810348    TRUE
## ar1     -0.99999999  1.0000000 0.29238716    TRUE
## omega    0.00000100 100.0000000 0.10000000    TRUE
## alpha1   0.00000001  1.0000000 0.10000000    TRUE
## gamma1  -0.99999999  1.0000000 0.10000000    FALSE
## delta    0.00000000  2.0000000 2.00000000    FALSE
## skew     0.10000000 10.0000000 1.00000000    FALSE
## shape    1.00000000 10.0000000 4.00000000    FALSE
## Index List of Parameters to be Optimized:
## mu  ar1  omega alpha1
## 1    2    3    4
## Persistence:          0.1
##
##
## --- START OF TRACE ---
## Selected Algorithm: nlminb
##
```

```

## R coded nlminb Solver:
##
## 0:      540.68757: 0.0581035 0.292387 0.100000 0.100000
## 1:      259.25047: 0.0581586 0.351171 1.04735 0.414755
## 2:      258.16987: 0.0582028 0.456570 1.03558 0.402981
## 3:      252.49959: 0.0574411 0.545312 0.727946 0.684987
## 4:      250.06907: 0.0574976 0.428738 0.487576 0.613163
## 5:      249.49697: 0.0730634 0.539003 0.542145 0.585597
## 6:      249.37315: 0.0770830 0.479971 0.564196 0.647037
## 7:      249.26377: 0.0814361 0.486281 0.508052 0.660433
## 8:      249.22636: 0.0809935 0.488008 0.527309 0.622664
## 9:      249.22557: 0.0800861 0.491156 0.531795 0.627067
## 10:     249.22447: 0.0791458 0.490668 0.531751 0.621237
## 11:     249.22437: 0.0788238 0.489868 0.531039 0.621091
## 12:     249.22436: 0.0788350 0.490084 0.531231 0.620527
## 13:     249.22436: 0.0788424 0.490071 0.531222 0.620629
## 14:     249.22436: 0.0788416 0.490071 0.531221 0.620624
##
## Final Estimate of the Negative LLH:
## LLH: -104.6908      norm LLH: -0.5598439
##      mu      ar1      omega      alpha1
## 0.01187989 0.49007057 0.01206114 0.62062414
##
## R-optimhess Difference Approximated Hessian Matrix:
##      mu      ar1      omega      alpha1
## mu      -12197.74425 -168.269073   -7034.6225    77.646879
## ar1      -168.26907 -234.866231    -192.8231    -6.463916
## omega    -7034.62251 -192.823054  -371223.5397  -1802.624159
## alpha1     77.64688   -6.463916   -1802.6242   -32.483982
## attr("time")
## Time difference of 0.006031036 secs
##
## --- END OF TRACE ---
##
## Time to Estimate Parameters:
## Time difference of 0.02100396 secs
summary(model)

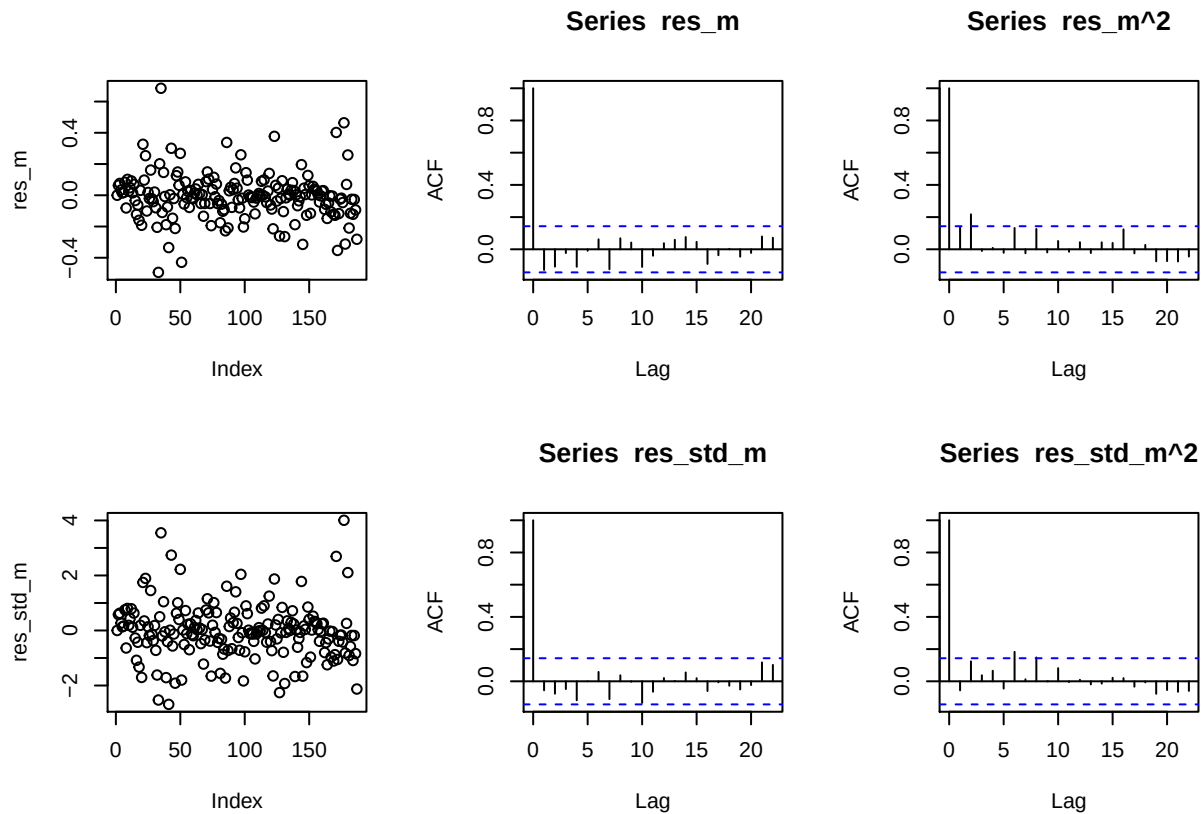
##
## Title:
## GARCH Modelling
##
## Call:
## garchFit(formula = ~arma(1, 0) + garch(1, 0), data = del.log.tbill)
##
## Mean and Variance Equation:
## data ~ arma(1, 0) + garch(1, 0)
## <environment: 0x7fe18fe5b310>
## [data = del.log.tbill]
##
## Conditional Distribution:
## norm
##

```



```
## Coefficient(s):
##      mu      ar1      omega      alpha1
## 0.011880 0.490071 0.012061 0.620624
##
## Std. Errors:
## based on Hessian
##
## Error Analysis:
##      Estimate Std. Error t value Pr(>|t|)
## mu      0.011880   0.009373   1.267  0.20500
## ar1     0.490071   0.065891   7.438 1.03e-13 ***
## omega   0.012061   0.001961   6.150 7.76e-10 ***
## alpha1  0.620624   0.210910   2.943  0.00325 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Log Likelihood:
## 104.6908      normalized:  0.5598439
##
## Description:
## Wed Nov 20 16:56:35 2019 by user:
##
##
## Standardised Residuals Tests:
##
##      Statistic p-Value
## Jarque-Bera Test  R    Chi^2 45.79453 1.137217e-10
## Shapiro-Wilk Test R    W      0.9540278 9.29422e-06
## Ljung-Box Test   R    Q(10) 11.76569 0.3010438
## Ljung-Box Test   R    Q(15) 13.38437 0.5726355
## Ljung-Box Test   R    Q(20) 14.94135 0.7797538
## Ljung-Box Test   R^2  Q(10) 17.25489 0.06891087
## Ljung-Box Test   R^2  Q(15) 17.49449 0.2901728
## Ljung-Box Test   R^2  Q(20) 19.69902 0.4768933
## LM Arch Test     R    TR^2  12.22925 0.4274482
##
## Information Criterion Statistics:
##      AIC      BIC      SIC      HQIC
## -1.076907 -1.007792 -1.077797 -1.048902
```

```
res_m = residuals(model)
res_std_m = res_m/model@sigma.t
par(mfrow = c(2, 3))
plot(res_m)
acf(res_m)
acf(res_m^2)
plot(res_std_m)
acf(res_std_m)
acf(res_std_m^2)
```



By looking at $\text{acf}(\text{res_m})$ and $\text{acf}(\text{res_m}^2)$, we did not find a strong evidence of correlation in further lags. We did not find strong evidence in volatility clustering either. By looking at scatter plots, there's no heteroskedasticity in residuals any longer. Thus, we may conclude working with log of Tbill is a better choice.

Part II (a)

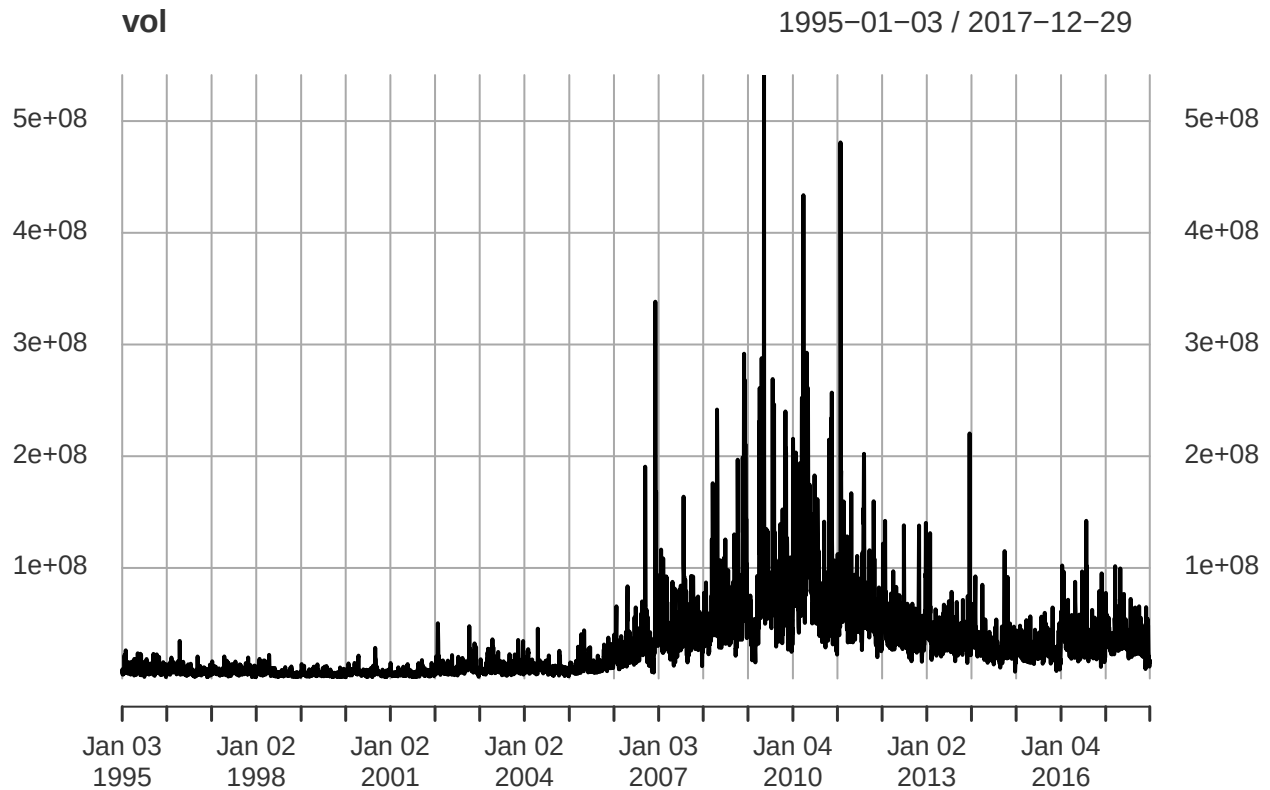
```
library(quantmod)
```

```
## Loading required package: xts
## Loading required package: zoo
##
## Attaching package: 'zoo'
## The following object is masked from 'package:timeSeries':
##
##   time<-
## The following objects are masked from 'package:base':
##
##   as.Date, as.Date.numeric
## Loading required package: TTR
##
## Attaching package: 'TTR'
## The following object is masked from 'package:fBasics':
##
##   volatility
## Version 0.4-0 included new data defaults. See ?getSymbols.
```

```
F_ = getSymbols("F", from = "1995-1-1", to = "2017-12-31", auto.assign = F)
```

```
## 'getSymbols' currently uses auto.assign=TRUE by default, but will
## use auto.assign=FALSE in 0.5-0. You will still be able to use
## 'loadSymbols' to automatically load data. getOption("getSymbols.env")
## and getOption("getSymbols.auto.assign") will still be checked for
## alternate defaults.
##
## This message is shown once per session and may be disabled by setting
## options("getSymbols.warning4.0"=FALSE). See ?getSymbols for details.
##
## WARNING: There have been significant changes to Yahoo Finance data.
## Please see the Warning section of '?getSymbols.yahoo' for details.
##
## This message is shown once per session and may be disabled by setting
## options("getSymbols.yahoo.warning"=FALSE).
```

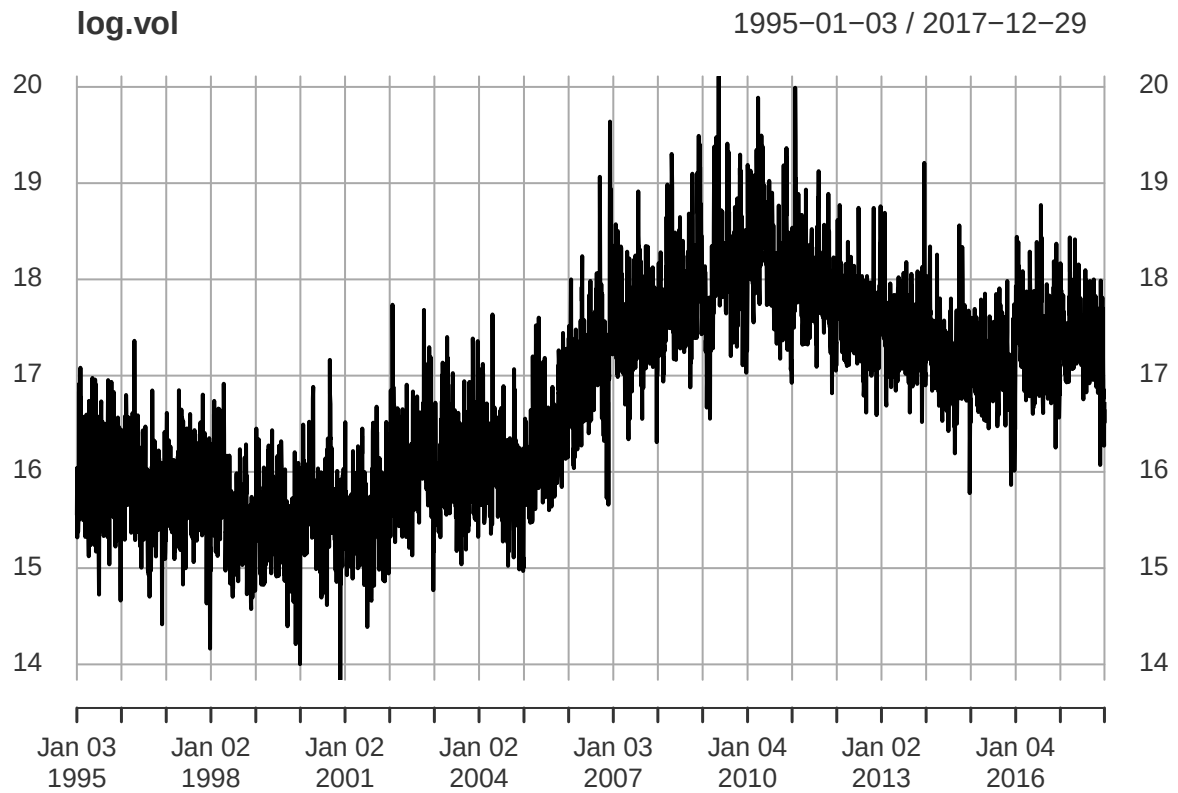
```
vol = Vo(F_)
plot(vol)
```



We noticed that as value goes up, variance goes up. So it would be better to take a log.

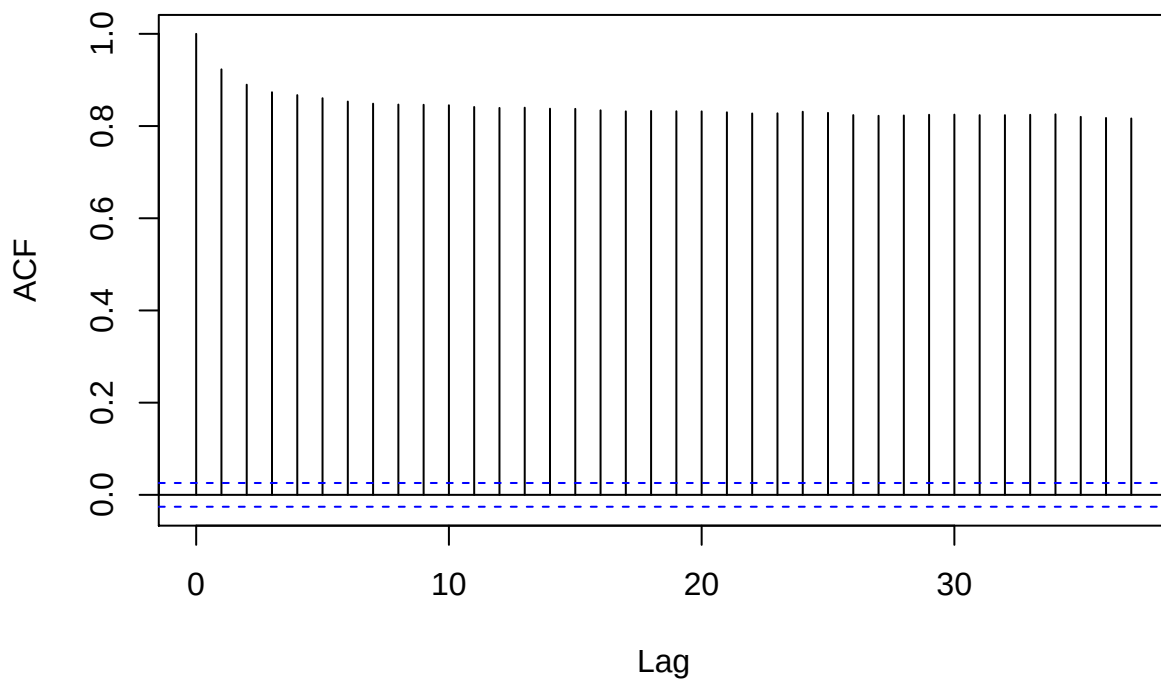
(b)

```
log.vol=log(vol)
plot(log.vol)
```



```
acf(log.vol)
```

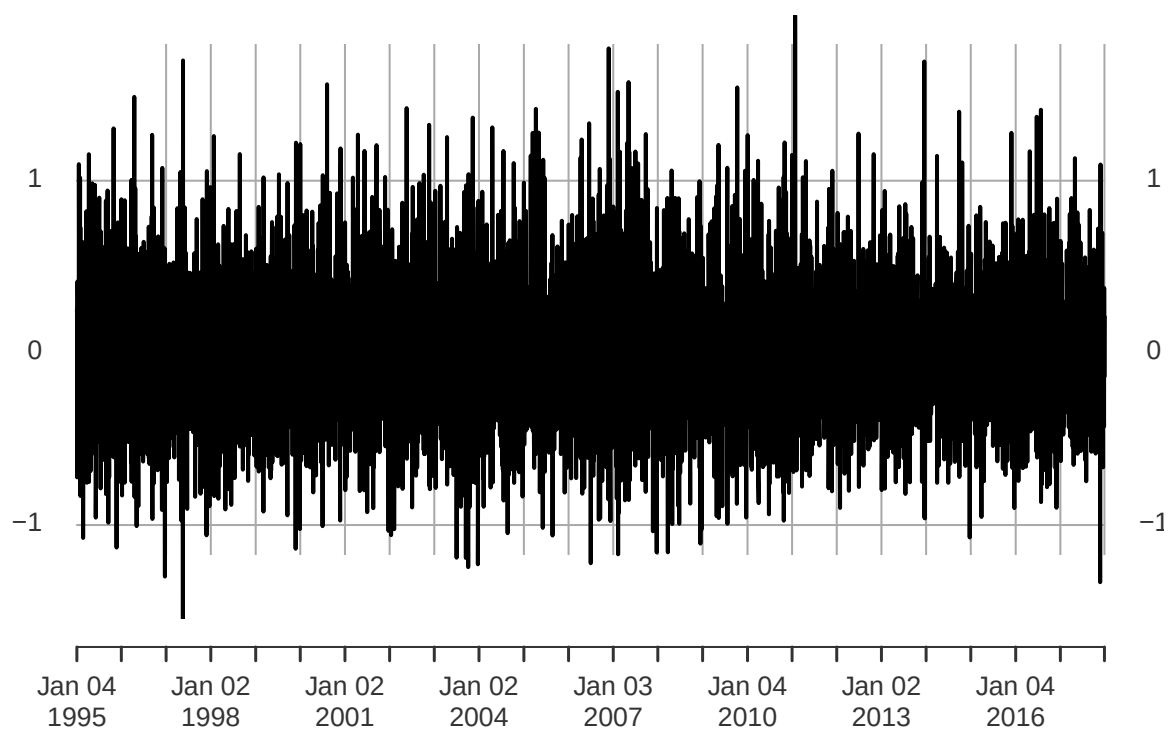
Series log.vol



```
diff.logvol = diff(log(vol), 1, 1)[-1]  
plot(diff.logvol)
```

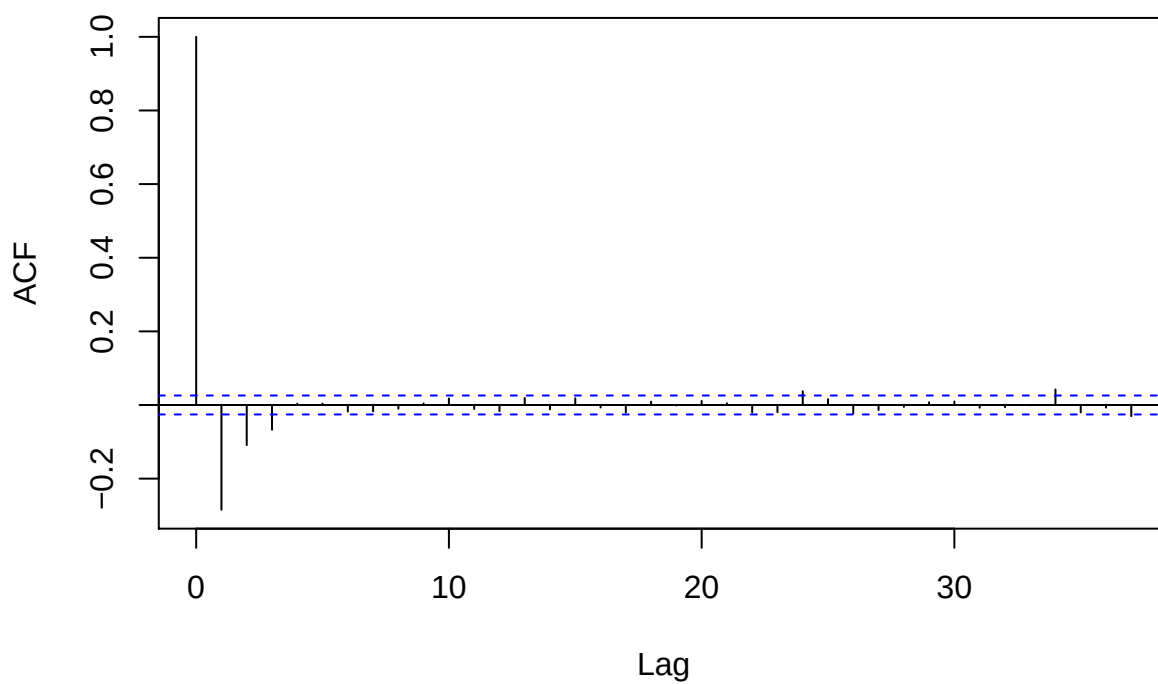
diff.logvol

1995-01-04 / 2017-12-29



```
acf(diff.logvol)
```

Series diff.logvol



```
adf.test(diff.logvol)
```

```
## Warning in adf.test(diff.logvol): p-value smaller than printed p-value
```

```
##
```

```
## Augmented Dickey-Fuller Test
```

```
##
```

```
## data: diff.logvol
```

```
## Dickey-Fuller = -27.019, Lag order = 17, p-value = 0.01
```

```
## alternative hypothesis: stationary
```

By looking at time series plot of both $\log(\text{vol})$ and $\text{acf}(\log(\text{vol}))$, I believe taking log of volume does not make it stationary. Taking first order differences does help. ACF plot suggests strong correlation in lag one two and three but not further lags. ACF test on the same time series gives a p-value of 0.01, which is less than 0.05, showing no strong evidence of nonzero correlation.

(c) i

```
holdfit = auto.arima(diff.logvol, approx = F, stepwise = F)
```

```
summary(holdfit)
```

```
## Series: diff.logvol
```

```
## ARIMA(2,0,2) with zero mean
```

```
##
```

```
## Coefficients:
```

```
##          ar1          ar2          ma1          ma2
```

```
##          1.2605   -0.3352   -1.7465    0.7507
```

```
## s.e.    0.0416    0.0283    0.0375    0.0368
```

```
##
```

```
## sigma^2 estimated as 0.1243: log likelihood=-2177.11
```

```
## AIC=4364.21   AICc=4364.22   BIC=4397.53
```

```
##
```

```
## Training set error measures:
```

```
##              ME          RMSE          MAE          MPE          MAPE          MASE
```

```
## Training set 0.004156084 0.3523726 0.2698589 42.48905 272.0876 0.5382601
```

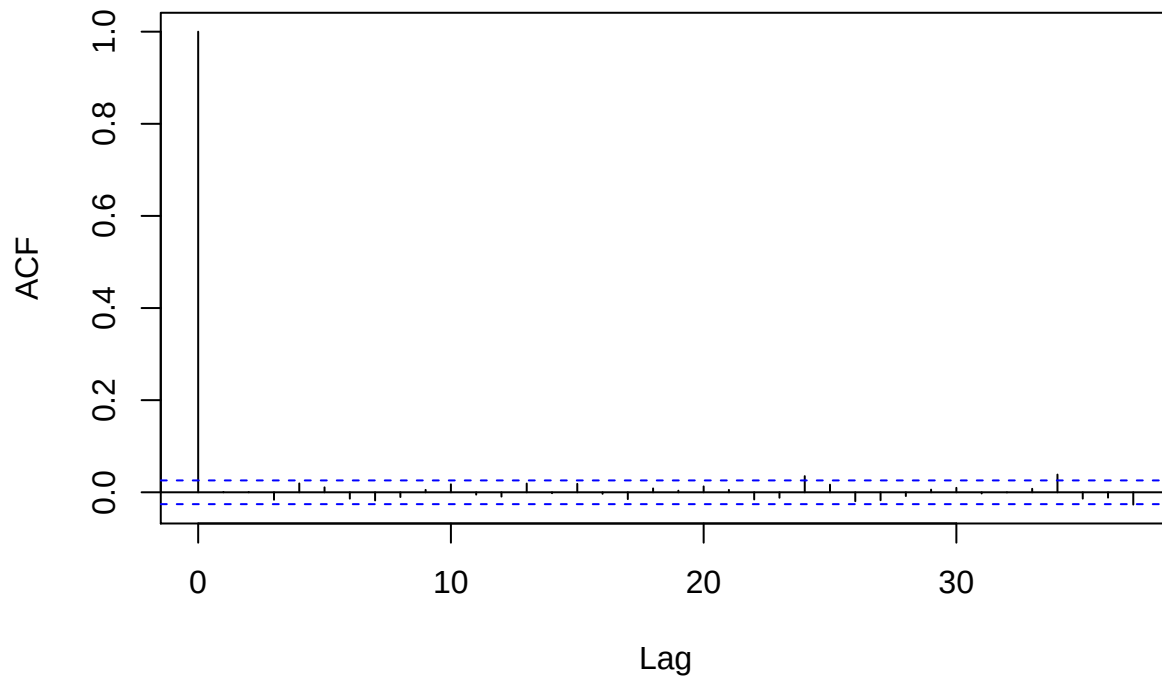
```
##              ACF1
```

```
## Training set 0.0007541923
```

(c) ii

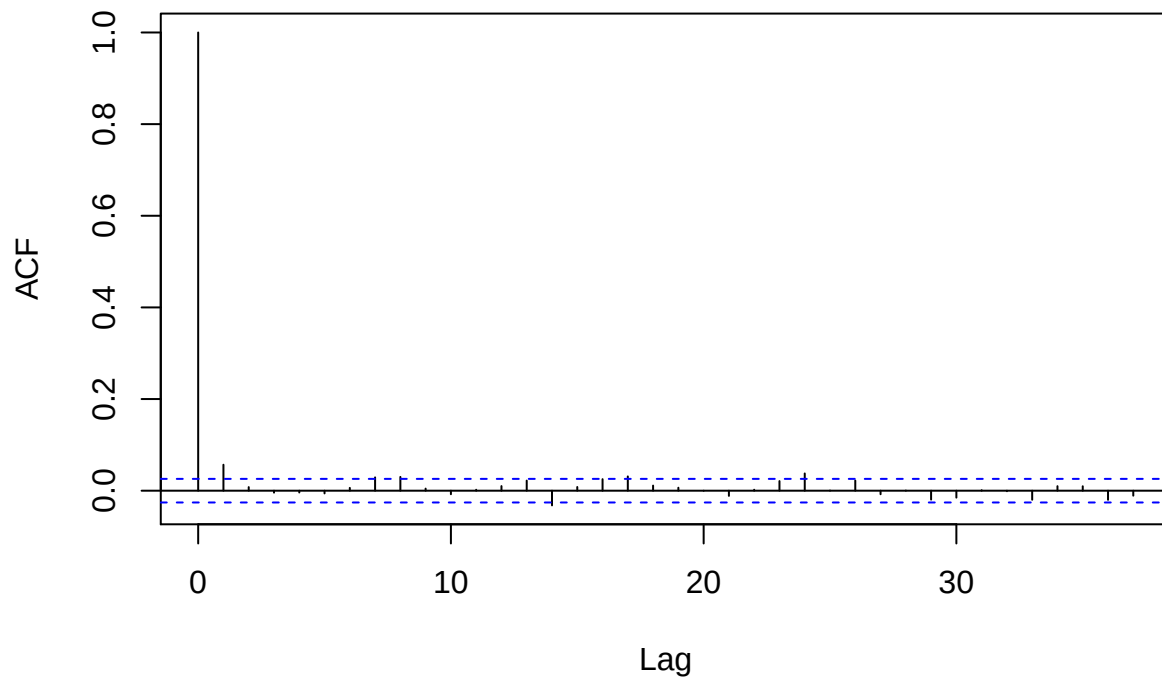
```
acf(holdfit$residuals)
```

Series holdfit\$residuals



```
acf(holdfit$residuals^2)
```

Series holdfit\$residuals^2



acf(res) indicates no correlation in further lags, which implies the choice of good model parameters. acf(res^2) shows no sign of volatility clustering.

(c) iii

```
garch_model = garchFit(formula = ~arma(2, 2) + garch(1, 1), diff.logvol)
```

```
## Warning in sqrt(diag(fit$cvar)): NaNs produced
```

```
summary(garch_model)
```

```
##
## Title:
##  GARCH Modelling
##
## Call:
##  garchFit(formula = ~arma(2, 2) + garch(1, 1), data = diff.logvol)
##
## Mean and Variance Equation:
##  data ~ arma(2, 2) + garch(1, 1)
## <environment: 0x7fe18ca7be60>
##  [data = diff.logvol]
##
## Conditional Distribution:
##  norm
##
## Coefficient(s):
##           mu           ar1           ar2           ma1           ma2
## -1.1309e-05  5.1546e-01  3.2310e-02 -1.0000e+00  4.0012e-02
##           omega        alpha1        beta1
##  1.1310e-01  4.6413e-02  4.7694e-02
##
## Std. Errors:
##  based on Hessian
##
## Error Analysis:
##           Estimate  Std. Error  t value Pr(>|t|)
## mu      -1.131e-05  1.868e-04  -0.061 0.951721
## ar1      5.155e-01      NA      NA      NA
## ar2      3.231e-02      NA      NA      NA
## ma1     -1.000e+00      NA      NA      NA
## ma2      4.001e-02      NA      NA      NA
## omega    1.131e-01  3.376e-02  3.350 0.000807 ***
## alpha1   4.641e-02  1.242e-02  3.736 0.000187 ***
## beta1    4.769e-02  2.717e-01  0.176 0.860638
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Log Likelihood:
##  -2183.891    normalized:  -0.3771833
##
## Description:
##  Wed Nov 20 16:56:48 2019 by user:
##
## Standardised Residuals Tests:
##           Statistic p-Value
## Jarque-Bera Test  R    Chi^2 631.5392 0
```



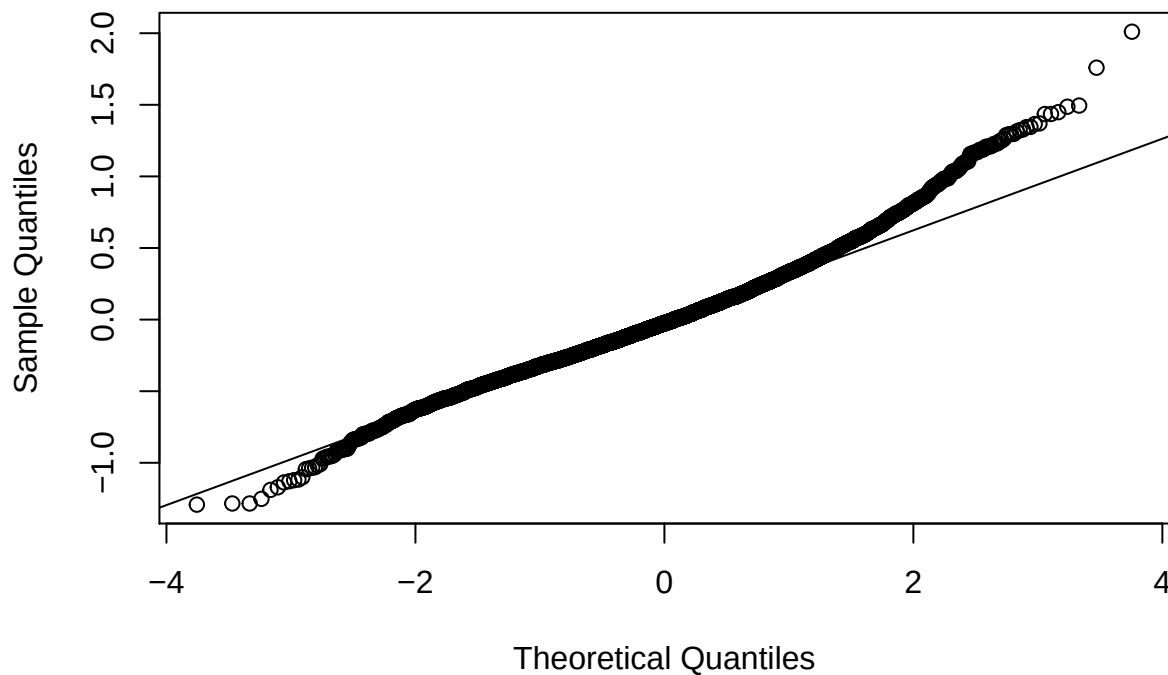
```
## Shapiro-Wilk Test R W NA NA
## Ljung-Box Test R Q(10) 25.21002 0.004961318
## Ljung-Box Test R Q(15) 28.81211 0.01700432
## Ljung-Box Test R Q(20) 32.2774 0.04043068
## Ljung-Box Test R^2 Q(10) 6.668117 0.7563611
## Ljung-Box Test R^2 Q(15) 16.06109 0.3780033
## Ljung-Box Test R^2 Q(20) 25.84002 0.1711606
## LM Arch Test R TR^2 7.978098 0.7868396
##
## Information Criterion Statistics:
## AIC BIC SIC HQIC
## 0.7571299 0.7663374 0.7571261 0.7603331
```

(c) iv: AIC goes up from 4364.21 to $0.7571299 \times 5790 = 4383.782$. So including the GARCH(1,1) component in the model was useful.

(d) v: Normality assumption is not justified, since the qq plot of residuals shows a way heavier tail than ordinary normal.

```
qqnorm(garch_model$residuals)
qqline(garch_model$residuals)
```

Normal Q-Q Plot



(c) vi:

```
garch_model_ = garchFit(formula = ~arma(2, 2) + aparch(1, 1), diff.logvol)
```

```
## Warning in sqrt(diag(fit$cvar)): NaNs produced
```

```
summary(garch_model_)
```

```
##
## Title:
## GARCH Modelling
```

```

##
## Call:
## garchFit(formula = ~arma(2, 2) + aparch(1, 1), data = diff.logvol)
##
## Mean and Variance Equation:
## data ~ arma(2, 2) + aparch(1, 1)
## <environment: 0x7fe192087790>
## [data = diff.logvol]
##
## Conditional Distribution:
## norm
##
## Coefficient(s):
##      mu      ar1      ar2      ma1      ma2
## 6.2158e-06 5.1253e-01 3.3131e-02 -1.0000e+00 4.0133e-02
##      omega    alpha1    gamma1    beta1    delta
## 1.0744e-01 4.1715e-02 -1.6858e-01 9.4936e-02 2.0000e+00
##
## Std. Errors:
## based on Hessian
##
## Error Analysis:
##      Estimate Std. Error t value Pr(>|t|)
## mu      6.216e-06 1.796e-04 0.035 0.972
## ar1      5.125e-01      NA      NA      NA
## ar2      3.313e-02      NA      NA      NA
## ma1     -1.000e+00      NA      NA      NA
## ma2      4.013e-02      NA      NA      NA
## omega    1.074e-01 2.315e-02 4.640 3.48e-06 ***
## alpha1   4.172e-02 1.018e-02 4.100 4.14e-05 ***
## gamma1  -1.686e-01      NA      NA      NA
## beta1     9.494e-02      NA      NA      NA
## delta    2.000e+00      NA      NA      NA
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Log Likelihood:
## -2183.109    normalized: -0.3770482
##
## Description:
## Wed Nov 20 16:56:51 2019 by user:
##
## Standardised Residuals Tests:
##
##      Statistic p-Value
## Jarque-Bera Test R Chi^2 627.35 0
## Shapiro-Wilk Test R W NA NA
## Ljung-Box Test R Q(10) 24.98481 0.005374365
## Ljung-Box Test R Q(15) 28.4841 0.01872786
## Ljung-Box Test R Q(20) 32.01174 0.04317341
## Ljung-Box Test R^2 Q(10) 6.60608 0.7620361
## Ljung-Box Test R^2 Q(15) 16.01457 0.3810837
## Ljung-Box Test R^2 Q(20) 25.90906 0.1688365
## LM Arch Test R TR^2 7.91225 0.7919496

```

```
##
## Information Criterion Statistics:
##      AIC      BIC      SIC      HQIC
## 0.7575506 0.7690599 0.7575446 0.7615546
```

Since AIC multiplier this time is 0.7575506, which is slightly larger than the previous 0.7571299, but remains almost the same. So AIC does not justify this additional complexity in this model.